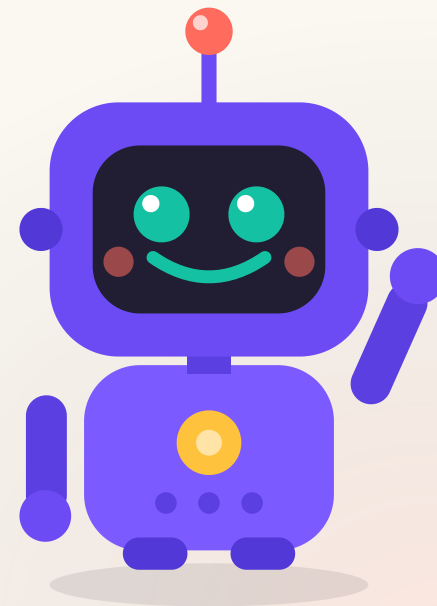


МОДУЛЬ 5 · УРОК 5.1

Помощники- функции

Один раз научим Бипа
делу — и будем звать
его сколько угодно раз.





Снова привет! Это я, Бип



Я ленивый робот: не люблю писать одно и то же дважды. Сегодня научу тебя собирать **помощников** — и работать станет легче!

Весь урок я рядом и подсказываю.

**Один раз написал —
сто раз позвал.**

Сегодня соберём команду помощников для школьных задач.

Вспомним прошлый урок

- Цикл `for` повторяет действия нужное число раз
- В цикле удобно перебирать числа и копить сумму
- `cin` спрашивает данные, `cout` выводит ответ



А теперь хитрость: научимся **не повторять** один и тот же код, а звать готового помощника по имени.



Что сегодня сделаем

- 1 Поймём, зачем нужны функции
- 2 Объявим своего первого помощника
- 3 Передадим ему данные — параметры
- 4 Научим его возвращать ответ — `return`
- 5 **Практика** — соберём помощников для школы
- 6 **Домашнее задание**

Зачем вообще нужны функции?

Представь: надо поздороваться с Аней, Борей и Викой.

main.cpp

```
std::cout << "Привет, Аня!\n";  
std::cout << "Привет, Боря!\n";  
std::cout << "Привет, Вика!\n";
```

✨ А ЗНАЕШЬ ЧТО?

Код почти одинаковый — меняется только имя. Когда копируешь строки **раз за разом** — пора звать помощника.

Функция — ЭТО ПОМОЩНИК

Дай ему задание один раз — и зови, когда нужно.



Вход

данные для работы



Работа

помощник делает дело



return

отдаёт ответ



Меньше кода — меньше ошибок. И программу куда легче читать!

Объявляем своего помощника

```
main.cpp

void pozdorovatsya() {
    std::cout << "Привет, класс!\n";
}
```

Это помощник по имени `pozdorovatsya`. Внутри `{ }` — что он делает.

СОВЕТ

void значит «помощник ничего не возвращает» — просто действует.

Как позвать помощника

```
main.cpp

int main() {
    pozdorovatsya();
    pozdorovatsya();
    return 0;
}
```

Имя со скобками `()` — это вызов. Позвали дважды — поздоровались дважды.

 ПОПРОБУЙ

Допиши третий вызов ``pozdorovatsya();`` — пусть Бип поздоровается трижды.

Разбираем по строчкам

```
main.cpp

void pozdorovatsya() {           // объявили помощника
    std::cout << "Привет!\n";    // что он делает
}

int main() {                     // главная функция
    pozdorovatsya();             // зовём помощника
}
```

💡 СОВЕТ

Функцию пиши **выше** `main` — чтобы `main` уже знал про помощника.

Параметры — данные для помощника

А если здороваться по имени? Передадим имя в скобках.



main.cpp

```
void privet(std::string imya) {  
    std::cout << "Привет, " << imya << "!\n";  
}
```



`std::string imya` в скобках — это **параметр**. При вызове ты кладёшь туда нужное имя, а я подставляю его в работу.

Зовём с разными именами

Код:

```
main.cpp  
  
privet("Аня");  
privet("Боря");
```

Результат:

► Привет, Аня!

► Привет, Боря!

Один помощник — а здоровается с кем угодно!

`return` — ПОМОЩНИК ВОЗВРАЩАЕТ ОТВЕТ

Функция умеет не только действовать, но и посчитать и вернуть результат.

main.cpp

```
int ploshad(int a, int b) {  
    return a * b;  
}
```

💡 СОВЕТ

Вместо **void** пишем тип ответа — здесь **int**. А **return** отдаёт результат обратно тому, кто звал.

Используем ответ функции

```
main.cpp

int main() {
    int s = ploshad(4, 3);
    std::cout << "Площадь: " << s;
    return 0;
}
```

Помощник посчитал `4 * 3`, вернул 12 — мы сохранили его в `s`.

► Площадь: 12

Частые ошибки



Забыл `()`

`privet` без скобок — это не вызов. Пиши `privet();`.



Забыл `return`

Функция с типом `int` обязана вернуть число.



Помощник ниже `main`

`main` не знает про него. Пиши функцию **выше**.

Проверь себя · что выведет программа? 🧠



main.cpp

```
int dvazhdy(int x) {  
    return x + x;  
}  
  
int main() {  
    std::cout << dvazhdy(5);  
}
```

Подумай 10 секунд...

✨ А ЗНАЕШЬ ЧТО?

Выведет **▶ 10** — помощник взял **x = 5** и вернул **5 + 5**.

Проверь себя · ещё раз 🧠



main.cpp

```
void krik(std::string s) {  
    std::cout << s << "!!!";  
}  
  
int main() {  
    krik("Ура");  
}
```

Что появится на экране?

✨ А ЗНАЕШЬ ЧТО?

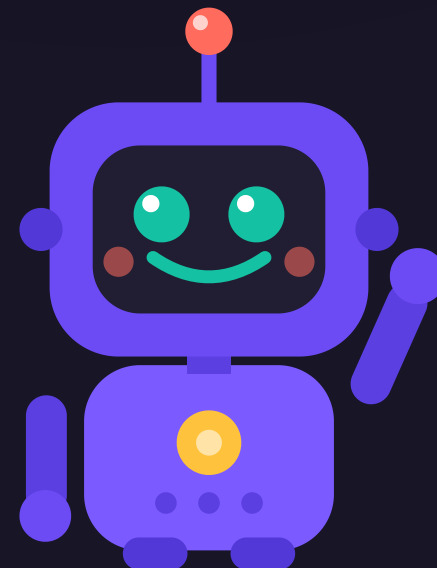
Выведет ► Ура!!! — помощник добавил три восклицательных знака к слову.

ПРАКТИКА В КЛАССЕ

Собираем ПОМОЩНИКОВ



Готовый код — на экране. Меняем под себя и запускаем!



✦ ЗАДАЧА

Задача 1 · Помощник-приветствие 🙌

Сделай функцию, которая здоровается по имени, и позови её трижды:



main.cpp

```
void privet(std::string imya) {
```

- В `main` позови `privet` с тремя именами одноклассников
- Запусти и проверь все три приветствия

Время: 5 минут

✦ ЗАДАЧА

Задача 2 · Площадь прямоугольника

Сделай функцию `ploshad`, которая считает площадь:



main.cpp

```
int ploshad(int a, int b) {
```

- Выведи площадь доски `4 × 3` и окна `2 × 5`
- Площадь = ширина × высота

Подсказка: ответ функции сразу клади в `std::cout`

✦ ЗАДАЧА

Задача 3 · Помощник-периметр

По образцу площади сделай функцию `perimetr(a, b)`, которая возвращает **периметр** прямоугольника.

- Формула периметра: $2 * (a + b)$
- Посчитай периметр для парты 120×60

Время: 6 минут

✦ ЗАДАЧА

Задача 4 · Школьный помощник

★ со звездочкой

Сделай функцию `summa3(a, b, c)`, которая возвращает сумму **трёх** оценок ученика.

- 1 Объяви функцию с тремя параметрами
- 2 Верни их сумму через `return`
- 3 Выведи сумму оценок `5 4 5`

Кто добавит ещё функцию для среднего балла – покажем классу!

Что мы сегодня узнали 💪



Теперь у тебя есть целая команда помощников. Пиши код один раз — а зови сколько влезет!

- Функция — помощник: пишем код один раз, зовём много раз
- Вызов функции — это её имя со скобками `()`
- Параметры в скобках передают данные внутрь
- `return` возвращает ответ, `void` — просто действует

Домашнее задание

1. Реши рабочий лист 5.1 — собери свою команду помощников.
2. Загляни в шпаргалку 5.1 — как объявить и позвать функцию.

Дальше: урок 5.2 — научим Бипа хранить целый список оценок класса.